Modular programs and proofs in Lean

Internship report
Arthur Adjedj
M2 MPRI, Université Paris-Saclay, ENS Paris-Saclay
Supervised by Yannick Forster in the Cambium team, INRIA Paris,

1 Summary

1.1 General context

Interactive Theorem Provers (ITPs), also known as proof assistants, are tools which allow for the development and verification of formal proofs. ITPs can be used to provide very strong gaurantees for software-in particular the complete absence of bugs-, which is why it's the tool of choice of several projects to verify real-world softare (Leroy [2009], AWS [2026]). Beyond their use for software verification, ITPs have been used to formalize landmark theorems in mathematics (Gonthier [2007], Gonthier et al. [2013]), and their use is on the rise in the mathematical community, with big developments (The mathlib Community [2020]) getting more and more public attention (Hartnett [2026]).

ITPs have particular issues regarding reusability of its definitions. For example, Adjedj et al. [2024] formalized a dependent type theory some years ago. Several projects have since sprouted from this work, forking the 30 thousand lines of code repository to extend on the type system formalized (Pédrot [2024], Laurent et al. [2024]), thus duplicating the codebase. In parallel, the initial repository got refactored to make the formalisation cleaner and easier to work on. Because of this code duplication, the benefits of the refactoring never reached the extensions, and would now take a lot of efforts to integrate into them.
The issue of modularity and code maintenance is not isolated to ITPs, and is coined as the **expression problem** (Wadler [1998]) in the sphere of programming languages:

> "The goal is to define a datatype by cases, where one can add new cases to the datatype and new functions over the datatype, *without recompiling existing code*."

Reusing and adapting existing data structures and programs modularly is a challenge for which few solutions have been produced over the years in industrial programming languages (Oliveira and Cook [2012]). The issue is exacerbated in ITPs, where in addition to programs, one needs to adapt proofs as well. In ITPs (which we focus on for the rest of the report), solutions range from making various encodings-such as Church encodings (Jansen [2013])-in which some notion of modularity comes internally "for free" (Delaware et al. [2013], Jamner et al. [2025]), to relying on meta-programming to achieve their goals (Forster and Stark [2020], Ebresafe et al. [2025]). Both approaches have had limited results until now, the former in part due to a lack of expressivity from the encodings used, the latter in part due to limitations in the meta-programming tools available during those developments. We discuss each tool's limitations in more depth in the related works (Section 7)

1.2 Research problem

The original expression problem from 1998 does not fit the constraints of modern ITPs, in particular with regards to avoiding recompiling existing code. Indeed, computers have since become fast enough, and memory storage big enough, that the constraints of compilation time or executable sizes are not central issues anymore. We instead focus our efforts on reducing the burden of

maintainance by providing better modularity in ITPs. As such, we focus on allowing users to extend datatypes by adding new constructors to them, as well as extending theorems and definitions which rely on them to hold for the new datatype. Such extensions should be doable on independent formalisations that were not made with our framework's constraints in mind.

1.3 Our contribution

We present **Gemel**[1], a new framework for writing modular code, written in the Lean 4 proof assistant. This library aims to provide a user-experience as similar to writing regular non-modular code as possible. This objective is achieved by avoiding the use of any internal encodings, manipulating Lean's concrete and abstract syntax trees directly, and making use of the powerful meta-programming capabilities of the language. **Gemel** declarations integrate all the features of regular Lean declaration such as dependent pattern-matching, tactic-based proofs and well-founded recursion. **Gemel** also allows extending a previous formalization that was not intended for extensions in mind, which we consider to be a crucial feature for the tool's adoption for real-world use-cases.

1.4 Arguments supporting its validity

To demonstrate the capacities of our system, we present two case-studies[2][3] implemented using **Gemel**.

The first takes an existing, independent formalization (Marmaduke [2026]) of the Simply-Typed Lambda Calculus (STLC), and extends it with new constructions, modularly extending key results of the original formalisation, such as strong normalisation, about the extended calculusi. Formalisations of STLC form a recurring case-study in previous works on modularity in ITPs, ours is the only one showing the capacity to build such extended formalisations "after the fact", i.e on an independent formalisation that was not made with modularity in mind.

The second case study consists of a construction of STLC and a Continuation-Passing Style (CPS) calculus, as well as a translation pass from the former to the latter. The formalisation then extends both STLC and CPS with various constructions, and extends the translation pass between the respective extensions. This reproduces the central case study produced by Jamner et al. [2025] in our own framework, without the need for any encodings.

Together, these two case-studies serve to show that the fundamentally basic ideas behind **Gemel** are powerful enough to scale up to all features provided by previous tools, as well as allowing one to to extend an independent formalisation after the fact, which is unique to our framework.

1.5 Summary and future work

We contribute a ready-to-use tool to easily extend formalisation. That tool is fairly complete in its original objective, as shown by our case-studies, though it exposes some limitations and a potential for further improvements. Improvements include the capacity to handle horizontal extensions, i.e the ability to add new fields to an existing constructor of a datatype, rather than just the ability to add new constructors entirely. A good next step would be to make the tool mature enough to be used in real world cases such as Lean's computer science library CSLib (Barrett et al. [2026]) , where e.g various type-systems formalisation could benefit from the tool.

[1] https://github.com/arthur-adjedj/Gemel

[2] https://github.com/arthur-adjedj/lean-stlc/

[3] https://github.com/arthur-adjedj/PyrosomeLeanCaseStudy/

2 Representation of modular syntax

We first examine existing solutions to producing modular code in ITPs, showing their respective strength and weaknesses, before explaining our framework for modularly extending syntax in **Gemel**.

2.1 Existing solutions

Solutions to the modularity problem in ITPs appear in two flavors: some tools work by encoding the datatypes manipulated in a framework which natively allows for some form of modularity (Delaware et al. [2013], Jamner et al. [2025]), others rely on meta-programming to achieve their goals.

Encodings have yet to become viable in dependently-typed ITPs, they incur an abstraction cost for users by leaking internal details, and cannot yet encode every datatypes that are native to such systems. AA: turn into paragraph

On the meta-programming side, Rocqet (Ebresafe et al. [2025]) and Rocq à la carte (Forster and Stark [2020]) both expose some additional syntax for the user to write in order to produce modular formalisms. AA: Full sentence for Rocqet Rocqet's design focuses on compiler verification specifically, whereas Rocq à la Carte aims to be a general framework for modular work, a scope that our project shares. We focus here on Rocq à la Carte's design here, with a deeper explanation of other frameworks in the Related Works (Section 7).

Rocq à la Carte's design centers on manipulating user-defined datatypes constructed through the merging of inductive functors:

```
inductive ExpVar (exp : Type) where
  | var : Nat → ExpVar exp

inductive ExpLam (exp : Type) where
  | app : exp → exp → ExpLam exp
  | lam : exp → ExpLam exp

inductive Exp₁ where
  | expvar : ExpVar Exp₁ → Exp₁
  | explam : ExpLam Exp₁ → Exp₁
```

Here, `ExpVar` and `ExpLam`, referred to as "feature functors", are parametrised by a type `exp`, and can be used to instantiate an inductive type with the desired features, `Exp₁` is such an example. Functions over types defined using feature functors can then be defined by first constructing said functions over the various feature functors, and composing them in an appropriate recursive function for the composed type:

```
def ExpLam.count (f : exp → Nat) : ExpLam exp → Nat
  | app e₁ e₂ => (f e₁) + (f e₂)
  | lam e => f e

def ExpVar.count (f : exp → Nat) : ExpVar exp → Nat
  | var _ => 1

def Exp₁.count : Exp₁ → Nat -- Error: Failed to show termination
  | expvar v => ExpVar.count Exp₁.count v
  | explam v => ExpLam.count Exp₁.count v
```

However, replicating (and extending on) this technique to bring modular tooling in the Lean proof assistant is not really feasable. The previous code snippet would work in Rocq, but throws a non-termination error in Lean. Indeed, functions in ITPs are required to be total for the sake of ensuring the soundness of the type-system (i.e the inability to prove `False`). Every popular ITP ensures this restriction differently (this is discussed more in depth in Section 4.2), but an important divergence between Rocq and Lean is that Rocq's system for detecting structural recursion works up to reduction, whereas Lean doesn't. This feature is paramount to Rocq à la Carte's method for producing modular code, and cannot be easily replicated in Lean due to divergences in the Lean's design decisions.

2.2 Our solution AA: preview of the user interface ?

One core objective of **Gemel** is the ability to extend a formalisation or program after the fact, no matter the shape it may have been given by a former implementor. In order to do so, we shift our focus from encodings "à la Carte" to making use of the existing metaprogramming APIs provided by Lean in order to achieve our goal of modularity. In particular, we want to be able to translate any definition or theorem written about one inductive type into one that instead mentions an extended version of said type. We refer to this translation function, written onwards as $[\![\_]\!]$, as a **modular map**.AA: talk conceptually about our solution first, technically later A term may get modularly mapped to something which contains holes, also known as metavariables. The reason why such holes may appear becomes more apparent in the next section. A modular map is dependent on a **mapping context**. This context maps some constants in the global environment to new terms. Using this mapping context, the algorithm for modular maps is straightforward: when modularly mapping a constant present in the mapping context, map said constant to the term it maps to. Otherwise, translate terms structurally (e.g $[\![\lambda a : A.t]\!] \coloneqq \lambda a : [\![A]\!].[\![t]\!]$). The following sections showcase how **Gemel** make use of this system of modular maps to achieve its goal of providing a user-friendly system of modular extensions.

3 Inductive Types and recursors

We hereon make use of our system of modular maps to provide users with ways to modularly extend previously defined programs and formalizations. From a user perspective, a Lean program mainly consists of defining new datatypes types, definitions and theorems. The main way to construct new datatypes in Lean is by defining new inductive types (Coquand and Paulin [1990]). This section focuses on how **Gemel** allows users to extend inductive definitions through addition of new constructors.

3.1 Inductive types in type theory

First, let us look at what an inductive type is. The general intuition of an inductive type is that it is a regular datatype, of which terms can be constructed by using recursors, and which can be eliminated through the use of a recursor, i.e an induction principle.
Inductive types are shaped as follows:

$$\textbf{inductive}\ \text{Ind}\ \overrightarrow{(p : P)} : \overrightarrow{(i : I)} \to \texttt{Type}\ \textbf{where}$$
$$\mid \textbf{ctor}_1 : \overrightarrow{(a_1 : A_1)} \to I\ \vec{p}\ \vec{d_1}$$
$$\dots$$
$$\mid \textbf{ctor}_n : \overrightarrow{(a_n : A_n)} \to I\ \vec{p}\ \overrightarrow{d_n}$$

An inductive type is composed of a number of different components: A list of parameters $\overrightarrow{(p : P)}$ which are uniform in that they stay the same in every occurrence of the type in its

constructors, a telescope of indices $\overrightarrow{(i : I)}$ that may vary in each occurrence, and a list of constructors for this type. Each constructor contains a list of fields living over the context formed by the parameters, and instantiates the indices of the inductive type they inhabit in a context containing those fields. Basic examples of inductive types include the natural numbers, lists, and vectors (i.e list indexed by their lengths):

```
inductive Nat : Type where
  | Z : Nat
  | S : Nat → Nat
```

```
inductive Vec (A : Type) : Nat → Type where
  | nil     : Vec A Z
  | cons {n}: A → Vec A n → Vec A (S n)
```

`Nat` is an inductive type with no parameters and no indices, as well as two constructors `Z` and `S`, the first one containing no field and the second containing one recursive occurrence of `Nat` as its sole field. In comparison, `Vec` is an inductive type which has one parameter `A`, is indexed by a natural number, and has two constructors. The former has no field and instantiate its index at 0, the second has 3 fields with one recursive one. `Vec A n` can be interpreted as the type of lists of `A` of size `n`.

To each inductive type is associated a recursor, i.e a way to eliminate a term of that type. For example, the recursor for `Nat` has the following type:
`Nat.rec` : $(\mathbf{motive} : \texttt{Nat} \to \texttt{Type}) \to \mathbf{motive}\ \texttt{Z} \to ((n : \texttt{Nat}) \to \mathbf{motive}\ n \to \mathbf{motive}\ (\texttt{S}\ n)) \to (t : \texttt{Nat}) \to \mathbf{motive}\ t$
The existence of that recursor can be interpreted as the statement that, given a predicate **motive** on natural numbers, an instance of that predicate on 0, and for every n, an instance of **motive** (`S` n) given **motive** n, that predicate holds for any natural number t. This corresponds exactly to the regular induction principle for natural numbers. Furthermore, recursors hold computational content in that they may reduce when applied to a constructor. For example, the two reduction rules for `Nat.rec` are as follows:

$$\texttt{Nat.rec P PZ PS Z} \rightsquigarrow \texttt{PZ}$$

$$\texttt{Nat.rec P PZ PS (S } n\texttt{)} \rightsquigarrow \texttt{PS } n \texttt{ (Nat.rec P PZ PS } n\texttt{)}$$

3.2 Inductive types in Gemel

With the general structure of an inductive type in mind, we can now consider ways in which one may extend an inductive type into another. **Gemel** provides a new Lean command which, given an inductive and new constructors, constructs another inductive type which extends the original one with these new constructors, adding the relevant mappings between from the first to the second. The syntax is as follows:

```
mod inductive <new inductive> extends <old inductive> where
  <new constructors>
```

Consider the following example: A user defines a notion of a variable type `Var`, which only has one constructor consisting of a natural number.

```
inductive Var where
  | var : Nat → Var
```

This representation for variables is common in both imperative and functional intermediate representations of programs. After producing an extensive API for this type, one may want to use this notion of variables as part of another type. Consider a new type `Term` which corresponds to that of a lambda-calculus. Such a calculus is constructed using variables, lambdas and applications. Such a type can be defined by extending the variable type as follows:

```
mod inductive Term extends Var where
  | lam : Term → Term
  | app : Term → Term → Term

#print Term
/- inductive Term : Type
   constructors:
     Term.lam : Term → Term
     Term.app : Term → Term → Term
     Term.var : Nat → Term -/
```

Similarly, one may want to extend the lambda-calculus with other constructs, such as booleans. Since `Term` is also just an inductive type, it can itself be extended with new constructions:

```
mod inductive BoolTerm extends Term where
  | true  : BoolTerm
  | false : BoolTerm
  --- if     c    then    a    else    b
  | ite : BoolTerm → BoolTerm → BoolTerm → BoolTerm

-- λ b => if b then false else true
#check lam (ite (var 0) false true) -- BoolTerm
```

Internally, after constructing this new type, **Gemel** adds the relevant mappings from the old type to the new one in the mapping context: Consider the previous example of `Var` and `Term`, their respective recursors are as follows:

```
Var.rec  : (motive : Var  → Type) → ((a : Nat) → motive (var a)) →
  (t : Var) → motive t
Term.rec : (motive : Term → Type) → ((a : Nat) → motive (var a)) →
  ((a : Term) → motive a → motive (lam a)) →
  ((f t : Term) → motive f → motive t → motive (app f t)) →
  (t : Term) → motive t
```

Consider a pretty-printing function for `Var`, defined explicitly using the type's recursor:

```
def Var.repr (t : Var) : String := Var.rec (fun _ => String) (fun x => s!"{x}") t
```

We see that the motive, existing minor for `var` and the major `t` can be translated into a `Term.rec` call as follows:

```
def Term.repr (t : Term) : String := Term.rec (fun _ => String) (fun x =>
s!"{x}") ?lam ?app t
```

Where `?lam` and `?app` are metavariables, i.e holes that still need to be resolved for the definition to be valid. These holes correspond to the branches for the `lam` and `app` cases respectively. The function can then be completed by giving a value to each hole:

```
def Term.repr (t : Term) : String := Term.rec (fun _ => String) (fun x => s!"{x}")
(fun _ sf => s!"λ {sf}")(fun _ _ sf st => s!"{sf} {st}") t
```

More generally, `Var.rec` can be modularly extended:
`⟦Var.rec motive pvar t⟧ => Term.rec ⟦ motive ⟧ ⟦ pvar ⟧ ?lam ?app ⟦ t ⟧`
Note that metavariable holes such as `?lam` and `?app` cannot, in general, be resolved automatically by Lean, and are instead meant to be resolved by the user, the next section on definitions extensions discusses the interface provided by **Gemel** for that purpose in more details.

To generalize the previous example, consider an inductive `A` of parameters $\overrightarrow{(p : P)}$, indices $\overrightarrow{(\imath : I)}$ and constructors $\overrightarrow{\mathbf{ctor}}$, as well as another type `B` with the same parameters and indices, as well as the same constructors + a new constructor **newctor**. Extending this to manage adding multiple constructors is trivial, we omit this detail here. We can map the type A to B and respectively every constructor of A to B's. The last missing piece to this translation is the recursor.

Recursors have the following shape:
$\text{A.rec} : \overrightarrow{(p : P)} \to \left(\mathbf{motive} : \overrightarrow{(\imath : I)} \to A\ \vec{p}\ \vec{\imath} \to \texttt{Type}\right) \to$
$\overrightarrow{(\mathbf{minor} : \ldots \to \mathbf{motive}\ (\mathbf{ctor}\ \ \ldots))} \to \overrightarrow{(\imath : I)} \to (a : A\ \vec{p}\ \vec{\imath}) \to \mathbf{motive}\ \vec{\imath}\ a$
where for every constructor of the type, there is a `minor` covering the (potentially recursive) case associated to it.
`B`'s recursor is very similar, except it contains an additional minor for the `newctor`'s case. **Gemel** modularly maps any application with head `A.rec` into `B.rec` by modularly mapping each argument (i.e the parameters, motive, minors and major), and adding a metavariable hole for the minor corresponding to `newctor`:

$$\left[\!\left[\text{A.rec}\ \vec{p}\ \mathbf{motive}\ \overrightarrow{\mathbf{minor}}\ \vec{\imath}\ a\right]\!\right] := \text{B.rec}\ \overrightarrow{[\![p]\!]}\ [\![\mathbf{motive}]\!]\ \overrightarrow{[\![\mathbf{minor}]\!]}\ ?m\ \overrightarrow{[\![\imath]\!]}\ [\![a]\!]$$

This mapping is automatically produced and added to the mapping context upon constructing a `mod inductive`.

Additionally to the primitives surrounding an inductive type, Lean also automatically produces more than a dozen auxiliary declarations, meant to make the automation and user-experience friendlier. For example, upon defining an inductive type `Foo`, Lean will, for every constructor `ctor`, define a helper lemma `Foo.ctor.inj` which proves the injectivity property of said constructor. In order to avoid making users define mappings between auxiliary declarations produced by the original and the extended type by hand, **Gemel** automatically generates said mappings automatically (e.g `Var.var.inj` gets automatically mapped to `Term.var.inj` in the previous example).

4 Pattern matching and recursion

Our framework now allows for inductive types to be extended through the addition of new constructors. This section focuses on how declarations (i.e definitions and theorems) can be extended and modularly mapped correctly.
Going back to the previous example of `Term` extending `Var`, consider a function `Var.repr`, this time written idiomatically using pattern-matching rather than a recursor:

```
def Var.repr : Var → String
  | var x => s!"#{x}"
```

Currently, a user would need to redo the `var` case in order to define a `Term.repr` function, e.g as follows:

```
def Term.repr : Term → String
  | var x => s!"#{x}"
  | app f x => s!"{Term.repr f} {Term.repr x}"
  | lam f => s!"λ {Term.repr f}"
```

This however leads to a duplication of the `var` case, which in turns lead to a burden of maintenance. If, for example, the user was to change how variables are pretty-printed in the `Var.var` case, this change would have to be copy-pasted for the `Term.var` case, which would be hard to track in general. Instead, **Gemel** introduces a new `mod def` syntax to allow users to reuse existing branches in a previously defined function, and only require them to fill-in the holes needed to go from a definition over one type to a definition of the same shape over an extension of said type. This syntax allows to rewrite `Term.repr` as follows:

```
mod def Term.repr extends Var.repr where
  extend with
  | app f x => s!"{Term.repr f} {Term.repr x}"
  | lam f => s!"λ {Term.repr f}"
```

Now, any change done to `Var.repr` would also change downstream of it for `Term.repr`.

The general elaboration process of that syntax is as follows. Given a declaration (e.g `Var.repr`), one may modularly map its body into a function over the extended type (here `Term`). Once that it done, users may be asked to fill in any remaining holes in the translated body to construct a well-formed declaration. Once that is done, the new declaration (here `Term.repr`) can be safely added to the global environment of declarations, and a mapping from `Var.repr` to `Term.repr` can be added to the mapping context.

In practice, implementing all of this in Lean has exposed some complications which necessitates careful design decisions, in particular with regards to how pattern-matching and recursion are to be managed. These two features are handled very differently by popular dependently-typed ITPs. In Rocq, matches and fixpoints are distinct primitive operations made explicit in the abstract syntax. Agda bundles these two notions in a single top-level primitive called a case-tree, where each function (and pattern-match present in a function) gets translated into a top-level case-tree. In both systems, termination-checking is a syntactic criteria that goes through a given fixpoint definition and checks that every recursive call in a function is ran on a strict subterm. Lean diverges from the tradition of having primitive operations, both for pattern-matching and for recursion, and elaborates both down to inductive recursor calls, following and extending on technics described in Goguen et al. [2006] and Cockx and Abel [2018]. For pattern-matching, it may appear as though Lean does have match expressions: they are part of the concrete syntax and get appropriately pretty-printed when looking back at the abstract syntax. Take for example a function of the form

```
def is_var_zero : Var → Bool
  | var 0     => true
  | var (n+1) => false
```

The shape of the match get elaborated into an auxiliary function:

```
is_var_zero.match_1 : (motive : Var → Type) → (x : Var) →
(Unit → motive (var 0)) → ((n : Nat) → motive (var (n + 1))) → motive x
```

The function is then applied with the right arguments to explicit the return type of the match-here called the `motive`-, the discriminant -here `x`-, and the right-hand-side of each branch:

```
def is_var_zero : Var → Bool := fun x =>
  is_var_zero.match_1 (fun _ => Bool) x (fun _ => true) (fun _ => false)
```

Note that a `Unit` argument appears in the `var 0` branch. This happens because Lean compiled code is call-by-value, which justifies lazily computing the branches of a given match.

For recursions, when a recursive function is defined, Lean tries to elaborate the function either into a structurally recursive term, using the recursor of whatever term decreases structurally in the recursive calls to write the function or tries to prove the function be well-founded, morally recursing over the accessibility predicate, similarly to Rocq's Equations (Sozeau and Mangin [2019]).

4.1 Extending pattern-matches

When it comes to extending matches, we are presented with two options. The first one consists of remarking that matches are simply encoded using recursors. As such, we may simply modularly map a given matcher and ask users to fill in the holes in it. This is unreasonable, as it exposes far too many internal details. The definition of a given match can quickly become complex if it matches on multiple elements, needs to generalize some variables or contains some proofs of equality between the thing matched on and a given constructor in a branch. Extending any of this would make an unreadable mess for the users and is too detached from what a user would do had he just written this as a normal definition. Consider the case of `is_var_zero` being extended to `Term`. For both the `app` and `lam` case, the result is the same (i.e `false`). Producing this extension would require both extending the implementation of `match_1` (by adding a branch for `(t : Term) → motive (lam t)` and `(f t : Term) → motive (app f t)`), and adding the right arguments for the right-hand-side of said new branches (i.e `fun t => false` and `fun f t => false`). This means, in this specific example, that a user would need to fill in 4 holes, half of them having a non-trivial shape which mentions an arbitrary `motive` that did not appear in the concrete syntax of the original definition. A second option for extending matches would be to remark that extending a match amounts to adding new branches to it such that it covers the relevant new constructors in the extended inductive type. As such, we may ask users to write down the relevant branches, in a syntax similar to how normal matches are written. This option turns out to be the most ergonomic, although it makes the implementation work measurably harder than the former. In order to accomodate for this feature, when modularly mapping the term of a given declaration, any application whose head is a matcher gets translated into a metavariable hole, saving the original shape of the expression along the way. When trying to solve that metavariable, the original shape of the matcher is reconstructed, similarly to how the pretty-printer handles this expression. We then elaborate the new match branches provided by the user, and re-elaborate this into a new match-expression. Note that the implementation does **not** do any anti-quotations (i.e it does not construct concrete syntax from the original abstract syntax), and instead manipulates both the abstract syntax of the old matcher and the concrete syntax of the new branches in parallel, using Lean's existing internals for elaborating matchers. The end-result is a legible syntax for extending past declarations.
Take the example of `is_var_zero` getting extended to `Term`:

```
def Term.is_var_zero extends is_var_zero where
  extend with
    | app _ _ => false
    | lam _   => false
```

First, the extension system of **Gemel** detects that `is_var_zero.match_1 (fun _ => Bool) x (fun _ => true) (fun_ => false)` is a matcher ap-

plication and stores the information about its shape (i.e what the motive, discriminants and existing branches are). All of this information gets modularly mapped to `Term` rather than the original `Var` (e.g `Var.var` become `Term.var` in the left-hand side of each branch). Then, the branches provided by the user (i.e `| app _ => false` and `| lam _ _ => false`) get elaborated as new branches to be added to the previous ones. This bundle of data is then passed out to the existing API Lean uses to elaborate normal pattern-matches, and produces a new match application that the old one then gets translated to.

4.2 Extending recursive functions

Though recursion is present at the level of the concrete syntax, it gets elaborated away into structural and well-founded recursion at the abstract syntax level, which means that functions internally get translated into things that are different in shape to a user's original definition. Because of this, directly modularly mapping the body of a recursive definition is not great, since it exposes internal encodings to the user, and asks one to manipulate those directly to extend the definition. Thankfully, Lean usually provides auxiliary lemmas that exhibit the original shape of the function that was defined. One of them, `foo.eq_def`, is a theorem of the form:

```
(a₁ : A₁) → ... → (aₙ : Aₙ)  → foo a₁ ... aₙ = <foo's definition>
```

As such, rather than use the original body of a definition, we can instead modular map the right-hand side of a function's `eq_def` whenever available, and then reuse the existing elaboration APIs Lean provides to translate such syntacticaly recursive functions to structurally recursive or well-founded functions. This in effect means a user can adapt the termination measure of an extended definition, relative to the original's, a feature no other extension system provides to our knowledge. Said feature is of particular importance when extending a non-recursive type to a recursive one, as is the case with the extension from `Var.repr` to `Term.repr`.

In particular, this allows us turn an originally non-recursive function into a recursive one if needed, as is the case with `Term.repr` extending `Var.repr`, or even change the termination measure that was originally used to adapt it to the new extended function.

4.3 Putting it all together

In practice, being able to change the proof of termination of an extended function compared to the original provides us with much more modularity, and is a feature we have not seen in any other project of the sort.
Furthermore, having matches be handled specially provides a clear distinction between the holes generated for matches and those generated by explicit uses of recursors, which usually only appear in proof terms generated by tactic calls such as `induction` and `cases`. branches. The complete syntax for `mod def` ends up as the following:

```
mod def <new function name> extends <old function name> where
  <match extension>
by
  <tactics>
<termination-information>
```

This in essence means our syntax for modular definitions offers a clear separation of concerns between proof holes, solved by tactics, and non-proof holes, solved by completing match The syntax asks for matchers to be completed first, before asking proof holes to be solved in a `by` block. The `by` block mainly only appears when extending theorems rather than definitions. Consider the following theorem:

```
theorem is_var_zero_eq (t : Var) (h : is_var_zero t) : t = var 0 := by
  cases t
  case var n =>
    cases n
    case zero => rfl
    case succ _ => nomatch h
```

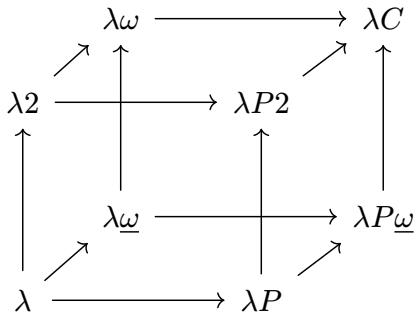
Extending it for `Term` will add two new proof holes originating from the `cases t`. These are given by the user in the `by` block as follows:

```
mod def Term.is_var_zero_eq extends is_var_zero_eq where
  by
  case lam _   => nomatch h
  case app _ _ => nomatch h
```

Termination information such as `termination_by` or `decreasing_by` can be additionally provided if needed to help with producing a well-founded recursive function. For example, the previous `Term.repr` function could have had an explicitly given annotation such as `termination_by structural t => t`, indicating that the function is structurally recursive over it's argument `t : Term`. In practice, and as is the case for the regular Lean definition, most functions' termination information is trivial enough to be inferred by the Lean, and thus does not require additional user-input.

5 Making extensions modular: modular merges

The current set-up allows one to extend previous definitions iteratively, though one may argue these extensions are not strictly "modular". In particular, one can currently only construct a declaration as an extension of a single other declaration rather than multiple. This, in particular, means one isn't able to compose different extensions. Take the example of the Barendregt lambda-cube (Barendregt [1991]):



The various vertices of the cube are extensions of the initial vertex corresponding to STLC, each adding new features to the original type theory. However, an important feature of this cube is that all extensions of λ can be written as mergings of its 3 adjactent vertices, namely λP, $\lambda 2$ and $\lambda\underline{\omega}$ (e.g $\lambda P2$ corresponds to the merging of $\lambda 2$ and λP). If one wanted to formalise each vertex of the cube before **Gemel**, they would need to write down 8 different formalisations. With our current framework, they need only write down 1 formalisation and 7 extensions of that base formalisation. If our system was truly modular however, one would only need to write down the base formalisation and the 3 adjacent extensions, the rest being simply generated by merging the adjacent extensions. To make this possible, we extend **Gemel**'s handling of inductive types and definitions extensions and describe a way to make them composable, thus achieving true modularity. In practice, from a user

perspective, all that gets added is the ability to define a `mod inductive` or `mod def` that `extends` more than one single declaration.

5.1 Inductive modularity

Extending `mod inductive` to be able to extend more than one inductive type is a fairly trivial task. Rather than only taking the constructors of one inductive, users may take constructors from multiple types. The mapping context will then map every old type to the new one, every old constructor from each old type to the relevant new ones. Consider the example of `BoolTerm` extending `Term` in Section 3.2, one may also extend `Term` to handle natural numbers, and merge the two extensions as follows:

```
mod inductive NatTerm extends Term where
  | zero : NatTerm
  | succ : NatTerm → NatTerm
  --   match n with | 0 => P0 | n+1 => Pn n
  | natmatch : NatTerm → NatTerm → NatTerm → NatTerm

mod inductive BoolNatTerm extends BoolTerm, NatTerm
```

The merging of constructors is nominal and type-driven: when merging `BoolTerm` and `NatTerm` together, **Gemel** remarks that the `var`, `app` and `lam` constructors are in common between the two, confirms their types are compatible, and does not duplicate them in the merging. As such, the generated inductive `BoolNatTerm` will contain all of the basic `Term` constructors, as well as the additional ones provided by `BoolTerm` and `NatTerm`, and maps the old inductives' constructions to the appropriate constructions of the new one.

5.2 Definition modularity

Consider the previous examples of defining `is_var_zero` and `is_var_zero_eq` to `BoolNatTerm`, consider the two declarations have already been extended to `BoolTerm` and `NatTerm`:

```
mod def BoolTerm.is_var_zero extends Term.is_var_zero where
  extend with
    | true | false | ite _ _ _ => false

mod def NatTerm.is_var_zero extends Term.is_var_zero where
  extend with
    | zero | succ _ | natmatch _ _ _ => false
```

We can modularly map both declarations into functions talking about `BoolNatTerm`. Both of them will be incomplete in that the pattern-match will be incomplete in both cases (e.g the cases for `zero` will be missing from the modular map of `BoolTerm.is_var_zero`). However, both declarations will have the same shape modulo having different holes in different places. We make use of that and construct an algorithm that syntactically merges two expressions together, and throws an error if their shape diverges. In practice, in the case of `is_var_zero`, this means the respective matches get merged correctly, meaning there is no need for the user to add anymore information in order to merge the two declarations:

```
-- No `extend with` block needed to complete the match
mod def BoolNatTerm.is_var_zero
  extends BoolTerm.is_var_zero, NatTerm.is_var_zero
```

Similarly, the proof for `is_var_zero_eq` need not anymore information after merging both `BoolTerm.is_var_zero_eq` and `NatTerm.is_var_zero_eq`:

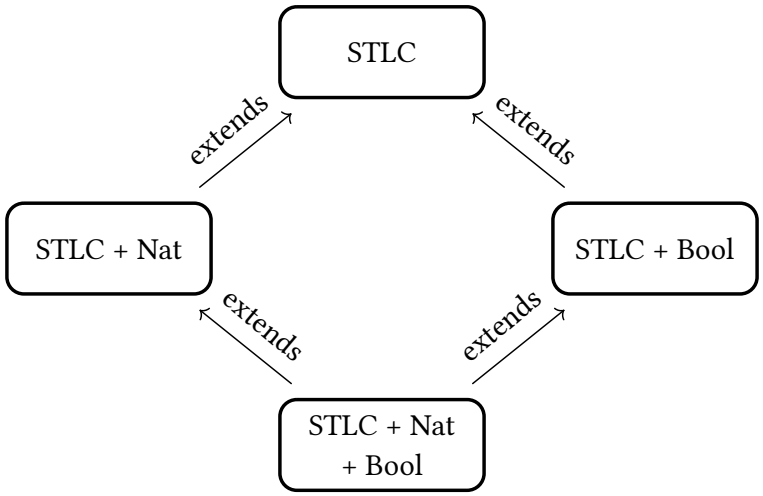
```
mod def BoolTerm.is_var_zero_eq extends Term.is_var_zero_eq where by
  ... -- proof omitted

mod def NatTerm.is_var_zero_eq extends Term.is_var_zero_eq where by
  ... -- proof omitted

-- no `by` block needed
mod def BoolNatTerm.is_var_zero_eq
  extends BoolTerm.is_var_zero_eq, NatTerm.is_var_zero _eq
```

6 Case studies

6.1 Strong normalisation of the Simply Typed Lambda Calculus



In order to iterate on this implementation and ensure its usability, we studied the case of taking an existing implementation of STLC (Marmaduke [2026]) which proves the strong normalization property of the system, and extended it with additional constructors, allowing us to modularly prove the strong normalization property of the extended system. In particular, the original normalization was **not** built with modularity in mind, and was still extendable after the fact. The original formalisation follows the usual proof of normalisation using a logical relation.

```
inductive Ty : Type where
| arrow : Ty → Ty → Ty
```

```
inductive Term where
| var : Nat → Term
| app : Term → Term → Term
| lam : Term → Term
```

```
inductive Typing : List Ty → Term → Ty → Type where
  | tyvar : Γ[n]? = some A → Typing Γ (var n) A
  | tyapp : Typing Γ f (arr A B) → Typing Γ t A → Typing Γ (app f t) B
  | tylam : Typing (A::Γ) t B → Typing Γ (lam t) B
```

We extend this base definition to add natural numbers.

```
mod inductive NatTerm.Ty
  extends Ty where
  | nat
```

```
mod inductive NatTerm.Term
  extends Term where
  | zero   : Term
  | succ   : Term → Term
  | natRec : Term → Term → Term → Term
```

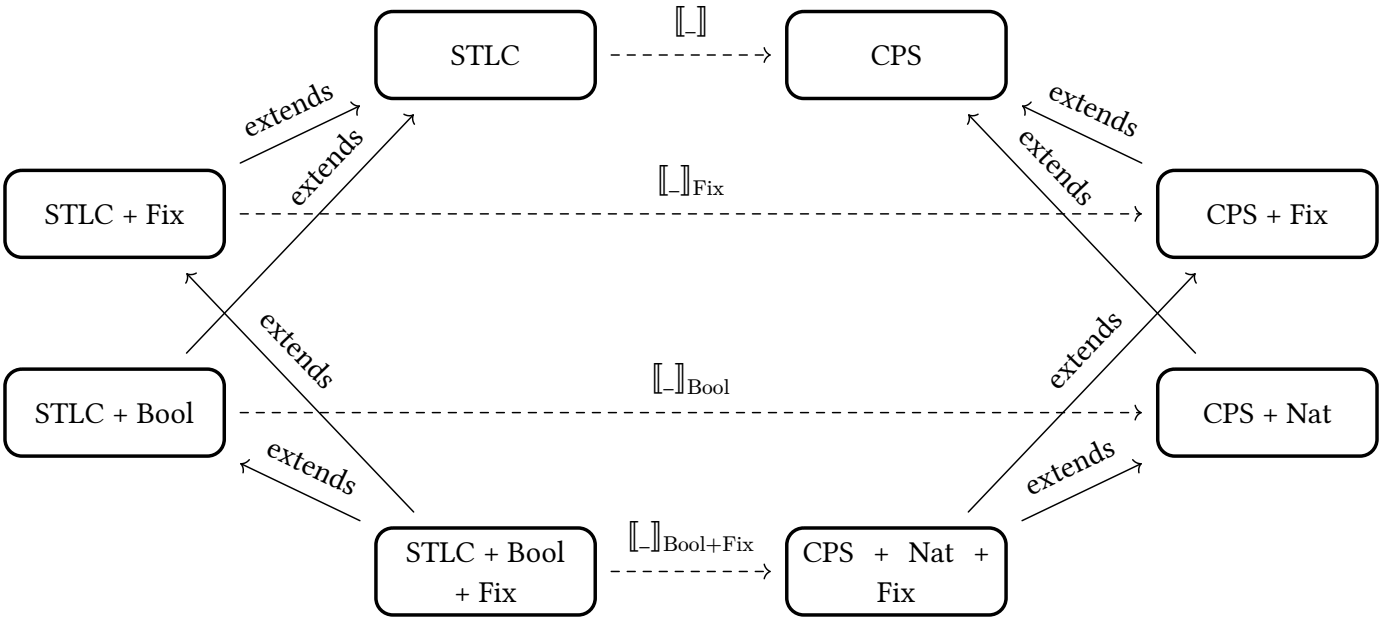
```
mod inductive NatTerm.Typing extends Typing where
  | zero   : Typing Γ zero nat
  | succ   : Typing Γ n nat → Typing Γ (succ n) .nat
  | natRec : Typing Γ n nat → Typing Γ P0 A
    → Typing Γ PS (arr nat (arr A A)) → Typing Γ (natRec n P0 PS) A
```

The total formalisation contains 12 inductive types as well as 92 definitions/theorems. Almost all of these were extended with no issue. The original formalisation contains a limitation to extensions in that the logical relation `LR` it defines to prove strong normalisation is too weak to prove the property on a system extended with natural numbers. We considered manually adding a modular map from `LR` to a variant strong enough for our purpose. However, the theorems in the original formalisation that make use of that LR rely heavily on its definitional equalities, which do not hold in a stronger formulation of the relation, meaning such a mapping leads to ill-typed terms when translating said theorems. As such, `LR` and the fundamental lemma had to be written as normal lean `def`s rather than as extensions. A solution to circumventing such issues, based on the idea of removing the reliance on some definitional equalities on a given term, is discussed in the future works.

To showcase the capacity to merge separate extensions, we produce another extension of STLC containing primitives for booleans, namely a boolean constructor in `Ty`, and term constructors for `true`, `false` and `if-then-else`. We then successfully construct a formalisation of STLC with both natural numbers and booleans by merging the two previous formalisations, thus proving the strong normalization of "STLC + Bool + Nat" fully modularly.

For lack of a better metric to measure the usability of **Gemel**, we remark that, while the original, independent formalisation of STLC was written in 1500 lines of code (LoC), the extensions for Nat and Bool respectively necessitated 1000 Loc, including the duplicated code about `LR`. The last extension, which had both Nat and Bool, and didn't duplicate any code since it already extended a powerful enough logical relation, was written in only 400 LoC in comparison.

6.2 STLC to CPS translation



To showcase the capabilities of our framework, and provide a point of comparison with another modular system, we reproduce the central case study shown in Pyrosome's (Jamner et al. [2025]) paper. This case study introduces two type system: STLC, and CPS-style type-system, as well as a translation from the former to the latter which proves $\beta\eta$-equivalence is conserved through that translation pass. The case-study then extends both STLC and CPS with new constructions (resp. booleans and a fixpoint operator for STLC, natural numbers and a fixpoint operator for CPS), and extends the translation pass accordingly. One feature of Pyrosome's case study is that their framework allows for limited forms of horizontal extensions, which our framework doesn't. In particular, STLC terms are separated between values (i.e terms whose head can't be anymore reduced) and expressions (who can reduce). This STLC formalisation differs from the first case study's in that typing is enforced intrinsically here, meaning the type of STLC terms are indexed by a context and a type (i.e `Term` has type `List Ty → Ty → Type`), whereas well-typedness is enforced extrinsically through a predicate in the former (i.e `Term` has type `Type` and a predicate `Typing: List Ty → Ty → Term → Prop` is defined). In order to modularly reuse as much API as possible between STLC, CPS and their respective extensions (such as substitution and renaming operations), both type systems are defined as extensions of a `Base` bundle, for which terms are just variables, and which defines a common API for both extensions to build off of. The shape of both systems differ in that both expressions and values are indexed by a type in STLC, but only values are in CPS. The type of terms is instead indexed by a context, a tag, and a dependent family `Tag.Data` over that tag which explicits by what data a term of a given tag is indexed by. As such, both STLC and CPS can extend the `Tag` type, and `Tag.Data` for their respective needs:

BASE

```
inductive Tag where
| val
inductive Ty where
| unit
def Tag.Data : Tag → Type
| val => Ty -- Value terms are always indexed by a type
inductive Term : List Ty → (t : Tag) → t.Data → Type
| var:(n : Fin Γ.length) → Γ[n] = A → Term Γ val
```

STLC

```
mod inductive Tag extends Base.Tag where
| exp
mod inductive Ty extends Base.Ty where
| arr (A B : Ty)

mod def Tag.Data extends Base.Tag.Data where
extends with
| exp => Ty --STLC exprs are indexed by a type
mod inductive Term extends Base.Term where
| ret : Term Γ val A → Term Γ exp A
| lam : Term (A::Γ) exp B → Term Γ val (.arr A B)
| app : Term Γ exp  (.arr A B) → Term Γ exp  A →
Term Γ exp B
```

CPS

```
mod inductive Tag extends Base.Tag where
| exp
inductive Ty where
| not   (A : Ty)
| times (A B : Ty)
mod def Tag.Data extends Base.Tag.Data where
extends with
| exp => Unit --CPS exprs are not
mod inductive Term extends Base.Term where
| lam : Term (A::Γ) exp () → Term Γ val A.not
| app : Term Γ val A.not → Term Γ val A → Term Γ
exp ()
| and_intro : Term Γ val A → Term Γ val B → Term
Γ val (A.times B)
| fst : Term Γ val (A.times B) → Term Γ val A
| snd : Term Γ val (Ty.times A B) → Term Γ val B
```

AA: This is very ugly and needs to be done better

7 Related works

We compare our approach with the existing solutions with the modularity problem in ITPs.

"Meta-Theory à la Carte" (Delaware et al. [2013]) and "Pyrosome" (Jamner et al. [2025]) base their modularity on internal encodings of types (namely, Mendler-style Church encodings in the former, and Generalized Algebraic Theories in the latter AA: Full sentence). In both cases, the constructions are inefficient, incapable of extending previously user-defined inductive types (*i.e.*, expecting users to rely on the aforementioned encodings from the ground up), and expose the underlying internals of the encodings to the user. Having to deal with encodings of types rather than types adds a heavy burden in particular for new users interested in program verification. The lesson to include inductive data-types natively has been learned early by popular ITPs (namely Rocq and Lean), who at first had no primitive notions of inductive types in their systems and then resorted to add those. Nowadays, most systems usually possess a syntactic notion of (co-)inductive types, and justify the ability to define these via some classes of models, allowing users to work with the abstractions these types provide, rather than with their encoding in said models. The only popular system that still relies on encodings to define such types, and manages to hide their implementation details well, is Isabelle/HOL.

On the other hand, Rocq à la Carte (Forster and Stark [2020]) relies on the meta-programming capabilities offered by the MetaRocq Project (Sozeau et al. [2020]) to allow users to construct new inductive types and functions by merging other inductive types, and/or adding new constructors. New functions on a "merged" datatype can then be constructed by merging past functions. The metaprogram then uses the given piece of information to reconstruct a new inductive type, and new functions, based on the information given by the user. While great for extending constructions

"vertically" (*i.e.,* by adding constructors to a type), this system does not allow for horizontal extensions (*i.e.,* extending the type signature of inductive types and their constructors). Furthermore, this approach has been hindered in the past by the lack of good metaprogramming frameworks in ITPs.

Rocqet comes the closest in philosophy to our implementation, relying heavily on metaprogramming and seemingly manipulating Rocq's AST directly. It however also doesn't allow for extensions after the fact, and their interface for extending past definitions is more restricted. Indeed, the only allows for constructing recursive functions with a single pattern-match with a single discriminant at the top of the function, where **Gemel** handles arbitrary pattern-matching. Furthermore, Rocqet is suited for defining and extending functions and proofs defined using structural induction on the datatype getting extended, **Gemel** on the other hand can handle structural and well-founded induction over arbitrary terms. Lastly, Rocqet requires users to explicit the motive function of each induction in their system. All of this comes at a cost for the user-experience.

None of these systems, independently of whether they use encodings or the meta-programming, handle all of the type-system of ITPs they are implemented for (*e.g.,* none handle co-inductive types), or allow users to extend past formalizations "after the fact". Instead, formalizations have to be built from the ground up with the expectation that they will be extended with a specific framework in mind, making them much less useful for real-world uses.

8 Future work

Gemel shows how directly manipulating the core AST of an ITP without relying on encodings can work for producing modular formalisations. There are however further design decisions that remain to be experimented with, and which may open up the door for further improvements to the tool.

There are morally two ways to extend an inductive type, referred to as vertical and horizontal extensions (Duggan and Sourelis [1996]). The former consists of adding new constructors to an inductive type, the latter of adding new fields to a type's constructor, or to the type itself. Most modular systems only allow for doing vertical extensions, as it is both the easiest one to justify and implement, and arguably the most useful one. Currently, **Gemel** only manages vertical extensions, though our second case-study (Section 6.2) shows how some horizontal extensionality can be recovered in the system through careful use of dependent types. The space of handling horizontal extensions has not been explored much in existing literature, the only system which, to our understanding, manages some form of horizontal extensions is Pyrosome. However, the scope of their extensions is much more limited and cares only about translation passes for compilers.

The central way in which extensions are added to the environment is by mapping constants to new (potentially incomplete) terms. For a translation using a constant found in the mapping context to work well, it is necessary that the reduction rules involving past constants also hold when mapping the constant to its associated term. This restriction is implicitly present when translating recursors and matches. However, as seen in the first case-study, it may sometimes be necessary for a formalisation to extend something (e.g the logical relation in the case study) with something possibly completely new, that may not hold the same definitional equalities. In order to do this, **Gemel** could, in the future, have some mechanism for transforming an expression which relies on some particular reduction rules/definitional equalities into one that does not. This idea of removing such rules is not new, and appears in works producing translations from Extensional Type Theory to Intensional Type Theory (Winterhalter [2020]), as well as works on removing

definitional equalities from Lean using a similar translation (Vaishnav [2025]). The latter work could in particular possibly be adapted to our needs since it is already implemented in Lean.

One challenge with extending independent formalisations is that shape of an original proof might not be fitting to prove an extended version of it. An important example of this issue is proofs by induction: it appeared a few times when testing **Gemel** that an original proof that was done by induction needed, for the extended proof to work, to generalize some variables when doing said induction, such that the induction hypotheses would be stronger. Similarly, it could happen that part of a proof originally just needed a case split where the new proof would require the full power of induction. However, the existing interface of metavariable holes morally only allows us to add the necessary content at the tips of our proof tree, not modify the proof-tree in its generality. To extend this framework, we could imagine allowing for some "after the fact" transformations, such as generalisations or turning a case split into an induction, to the rest of the proof tree, allowing for a more comfortable experience of doing extensions of independent formalisations.

References

Adjedj, A., Lennon-Bertrand, M., Maillard, K., Pédrot, P.M. and Pujet, L. (2024) Martin-Löf à la Coq, In. *Proceedings of the 13th ACM SIGPLAN International Conference on Certified Programs and Proofs*. CPP 2024, Association for Computing Machinery, London, UK 2024: pp. 230–245, doi:10.1145/3636501.3636951.

AWS (2026) Cedar Specification, 2026, https://github.com/cedar-policy/cedar-spec.

Barendregt, H. (1991) An Introduction to Generalized Type Systems, *Journal of Functional Programming*. 1991;1: 125–154., doi:10.1017/S0956796800020025.

Barrett, C., Chaudhuri, S., Montesi, F., et al. (2026) CSLib: The Lean Computer Science Library, *arXiv e-prints*. February 2026., doi:10.48550/arXiv.2602.04846.

Cockx, J. and Abel, A. (2018) Elaborating Dependent (Co)Pattern Matching, *Proc ACM Program Lang*. 2018;2(ICFP)., doi:10.1145/3236770.

Coquand, T. and Paulin, C. (1990) Inductively defined types, In. *COLOG-88*, Springer Berlin Heidelberg 1990: pp. 50–66, doi:10.1007/3-540-52335-9_47.

Delaware, B., S. Oliveira, B.C. d. and Schrijvers, T. (2013) Meta-theory à la carte, *SIGPLAN Not.* 2013;48(1): 207–218., doi:10.1145/2480359.2429094.

Duggan, D. and Sourelis, C. (1996) Mixin modules, *SIGPLAN Not.* 1996;31(6): 262–273., doi:10.1145/232629.232654.

Ebresafe, O., Zhao, I., Jin, E., Bright, A., Jian, C. and Zhang, Y. (2025) Certified Compilers à la Carte, *Proc ACM Program Lang*. 2025;9(PLDI)., doi:10.1145/3729261.

Forster, Y. and Stark, K. (2020) Coq à la carte: a practical approach to modular syntax with binders, In: Blanchette, J. and Hritcu, C. (eds.). *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2020, New Orleans, LA, USA, January 20-21, 2020*, ACM 2020: pp. 186–200, doi:10.1145/3372885.3373817.

Goguen, H., McBride, C. and McKinna, J. (2006) Eliminating Dependent Pattern Matching, In: Futatsugi, K., Jouannaud, J.P. and Meseguer, J. (eds.). *Algebra, Meaning, And Computation, Essays Dedicated to Joseph A. Goguen on the Occasion of His 65th Birthday*. Vol 4060. Lecture Notes in Computer Science, Springer 2006: pp. 521–540, doi:10.1007/11780274_27.

Gonthier, G. (2007) The Four Colour Theorem: Engineering of a Formal Proof, In: Kapur, D. (ed.). *Computer Mathematics, 8th Asian Symposium, ASCM 2007, Singapore, December 15-17, 2007. Revised and Invited Papers*. Vol 5081. Lecture Notes in Computer Science, Springer 2007: p. 333, doi:10.1007/978-3-540-87827-8_28.

Gonthier, G., Asperti, A., Avigad, J., et al. (2013) A Machine-Checked Proof of the Odd Order Theorem, In: Blazy, S., Paulin-Mohring, C. and Pichardie, D. (eds.). *Interactive Theorem Proving*, Springer Berlin Heidelberg, Berlin, Heidelberg 2013: pp. 163–179.

Hartnett, K. (2026) *The Proof in the Code*, Quanta Books 2026.

Jamner, D., Kammer, G., Nag, R. and Chlipala, A. (2025) Pyrosome: Verified Compilation for Modular Metatheory, *Proc ACM Program Lang.* 2025;9(OOPSLA2)., doi:10.1145/3763052.

Jansen, J.M. (2013) Programming in the \lambda-Calculus: From Church to Scott and Back, In: Achten, P. and Koopman, P. (eds.). *The Beauty of Functional Code: Essays Dedicated to Rinus Plasmeijer on the Occasion of His 61st Birthday*, Springer Berlin Heidelberg, Berlin, Heidelberg 2013: pp. 168–180, doi:10.1007/978-3-642-40355-2_12.

Kovács, A. (2020) Elaboration with first-class implicit function types, *Proc ACM Program Lang.* 2020;4(ICFP)., doi:10.1145/3408983.

Laurent, T., Lennon-Bertrand, M. and Maillard, K. (2024) Definitional Functoriality for Dependent (Sub)Types, In: Weirich, S. (ed.). *33rd European Symposium on Programming, ESOP 2024*. Vol 14576. Lecture Notes in Computer Science, Springer 2024: pp. 302–331, doi:10.1007/978-3-031-57262-3_13.

Leroy, X. (2009) Formal verification of a realistic compiler, *Communications of the ACM.* 2009;52(7): 107–115., doi:10.1145/1538788.1538814.

Marmaduke, A. (2026) lean-stlc, 2026, https://github.com/amarmaduke/lean-stlc.

The mathlib Community (2020) The Lean Mathematical Library, In. *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*. CPP 2020, Association for Computing Machinery, New Orleans, LA, USA 2020: pp. 367–381, doi:10.1145/3372885.3373824.

Oliveira, B.C. d. S. and Cook, W.R. (2012) Extensibility for the Masses, In: Noble, J. (ed.). *ECOOP 2012 – Object-Oriented Programming*, Springer Berlin Heidelberg, Berlin, Heidelberg 2012: pp. 2–27.

Pédrot, P.M. (2024) "Upon This Quote I Will Build My Church Thesis", In. *Proceedings of the 39th Annual ACM/IEEE Symposium on Logic in Computer Science*. LICS '24, Association for Computing Machinery, Tallinn, Estonia 2024, doi:10.1145/3661814.3662070.

Sozeau, M. and Mangin, C. (2019) Equations Reloaded: High-Level Dependently-Typed Functional Programming and Proving in Coq, *Proc ACM Program Lang.* 2019;3(ICFP)., doi:10.1145/3341690.

Sozeau, M., Anand, A., Boulier, S., et al. (2020) The MetaCoq Project, *Journal of Automated Reasoning*. 2020., doi:10.1007/s10817-019-09540-0.

Sozeau, M., Forster, Y., Lennon-Bertrand, M., Nielsen, J., Tabareau, N. and Winterhalter, T. (2025) Correct and Complete Type Checking and Certified Erasure for Coq, in Coq, *Journal of the ACM*. January 2025., doi:10.1145/3706056.

Vaishnav, R. (2025) Lean4Less: Eliminating Definitional Equalities from Lean via an Extensional-to-Intensional Translation, In. *Theoretical Aspects of Computing – ICTAC 2025 22nd International Colloquium*. Vol Lecture Notes in Computer Science, Marrakech, Morocco 2025, https://inria.hal.science/hal-05310102.

Wadler, P. (1998) The expression problem, November 1998, https://homepages.inf.ed.ac.uk/wadler/papers/expression/expression.txt.

Winterhalter, T. (2020) *Formalisation and Meta-Theory of Type Theory*, Theses, 2020, doi:10.70675/f3c96514z4373z4f99z8b3fzb1b41307d558.

A Formal presentation of modular maps

This section makes a partial attempt at justifying the implementation of modular maps in our system, and why it can be trusted to make well-formed terms.

First, let's define the syntax we'll be using in this toy presentation. This is a basic dependently-typed system equiped with a predicative hierarchy of universes à la Russel, as well as constants and metavariables. For the sake of simplifying the presentation, we will assume constants cannot be unfolded (i.e we will only care about β/η-reduction, not δ-reduction). This, in particular, does not give good justification as to why recursors of inductive types may be extended safely.

$$
\begin{aligned}
\text{Terms} : t, f, A, B \coloneqq\ & x && \text{(variable)} \\
\mid\ & d && \text{(constant)} \\
\mid\ & m && \text{(metavariable)} \\
\mid\ & f\ t && \text{(application)} \\
\mid\ & \lambda(x : A) \mapsto t && \text{(abstraction)} \\
\mid\ & (x : A) \to B && (\Pi\text{-type}) \\
\mid\ & \texttt{Type}_i && \text{(universe)}
\end{aligned}
$$

Listing 1: Syntax of our type theory

The usual typing judgements one would expect apply. These judgement need to carry not only the usual variable context Γ, but also a global constants context Σ to handle the type of constants, similarly to Sozeau et al. [2025], as well as a metavariable context Θ that holds, for each metavariable, both the context and the type of said metavariable, similarly to Kovács [2020]. The idea is that constants may be mapped to some term containing "holes", i.e metavariables, allowing us to make the modular maps.

We define a judgement $\Sigma \mid \Phi \mid \Theta \mid \Gamma \Rightarrow \Delta \vdash t \Rightarrow t' : A \Rightarrow A'$ encompassing the behaviour of the modular map in practice. This judgement carries the same contexts found in the aforementionned typing judgements, as well as a mapping context Φ, which maps constants to the bundling of a metavariable context and a term that may contain the metavariables present in the context it's bundled with. The judgement states tracks both the base variable context and a variable context for the modularly mapped term, as well as the type of both the original term and the modularly mapped one. The modular map acts structurally over the usual constructors of our type-theory, and relies on the mapping context to translate constants into their modular map. Furthermore, when modularly mapping a constant, the metavariable context it carries is weakened/lifted, such that every metavariable lives in some telescope over translated variable context.

$\boxed{\Sigma \mid \Phi \mid \Theta \mid \Gamma \Rightarrow \Delta \vdash t \Rightarrow t' : A \Rightarrow A'}$

(In global context Σ, mapping context Φ, metavariable context Θ, term t of type A in context Γ maps to term t' of type A' in context Δ)

$$\frac{}{\Sigma \mid \Phi \mid \Theta \mid \Gamma \Rightarrow \Delta \vdash \mathsf{Type}_i \Rightarrow \mathsf{Type}_i : \mathsf{Type}_{i+1} \Rightarrow \mathsf{Type}_{i+1}}$$

$$\frac{(x : A) \in \Gamma \qquad (x : A') \in \Delta}{\Sigma \mid \Phi \mid \Theta \mid \Gamma \Rightarrow \Delta \vdash x \Rightarrow x : A \Rightarrow A'}$$

$$\frac{((d : A) \mapsto (\Theta, t : A')) \in \Phi}{\Sigma \mid \Phi \mid \Uparrow_\Delta \Theta \mid \Gamma \Rightarrow \Delta \vdash d \Rightarrow t : A \Rightarrow A'}$$

$$\frac{(m : (\Gamma_2, A)) \in \Theta \quad \Gamma_2 \subset \Gamma_1 \quad \Delta_2 \subset \Delta_1 \quad \Sigma \mid \Phi \mid \Theta \mid \Gamma_2 \Rightarrow \Delta_2 \vdash A \Rightarrow A' : \mathsf{Type}_i \Rightarrow \mathsf{Type}_i}{\Sigma \mid \Phi \mid \Theta \mid \Gamma_1 \Rightarrow \Delta_1 \vdash m \Rightarrow m : A \Rightarrow A'}$$

$$\frac{\begin{array}{c}\Sigma \mid \Phi \mid \Theta_1 \mid \Gamma \Rightarrow \Delta \vdash f \Rightarrow f' : (x : A) \to B \Rightarrow (x : A') \to B' \\ \Sigma \mid \Phi \mid \Theta_2 \mid \Gamma \Rightarrow \Delta \vdash t \Rightarrow t' : A \Rightarrow A' \quad \mathrm{MV}(\Theta_1) \cap \mathrm{MV}(\Theta_2) = \varnothing\end{array}}{\Sigma \mid \Phi \mid \Theta_1 \cup \Theta_2 \mid \Gamma \Rightarrow \Delta \vdash f\ t \Rightarrow f\ t : B[x := t] \Rightarrow B'[x := t']}$$

$$\frac{\begin{array}{c}\Sigma \mid \Phi \mid \Theta_1 \mid \Gamma \Rightarrow \Delta \vdash A \Rightarrow A' : \mathsf{Type}_i \Rightarrow \mathsf{Type}_i \\ \Sigma \mid \Phi \mid \Theta_2 \mid (\Gamma, x : A) \Rightarrow (\Delta, x : A) \vdash B \Rightarrow B' : \mathsf{Type}_j \Rightarrow \mathsf{Type}_j) \quad \mathrm{MV}(\Theta_1) \cap \mathrm{MV}(\Theta_2) = \varnothing\end{array}}{\Sigma \mid \Phi \mid \Theta_1 \cup \Theta_2 \mid \Gamma \Rightarrow \Delta \vdash (x : A) \to B \Rightarrow (x : A') \to B' : \mathsf{Type}_{\max(i,j)} \Rightarrow \mathsf{Type}_{\max(i,j)}}$$

$$\frac{\begin{array}{c}\Sigma \mid \Phi \mid \Theta_1 \mid \Gamma \Rightarrow \Delta \vdash (x : A) \to B \Rightarrow (x : A') \to B' : \mathsf{Type}_i \Rightarrow \mathsf{Type}_i \\ \Sigma \mid \Phi \mid \Theta_2 \mid (\Gamma, x : A) \Rightarrow (\Delta, x : A) \vdash t \Rightarrow t' : B \Rightarrow B') \quad \mathrm{MV}(\Theta_1) \cap \mathrm{MV}(\Theta_2) = \varnothing\end{array}}{\Sigma \mid \Phi \mid \Theta_1 \cup \Theta_2 \mid \Gamma \Rightarrow \Delta \vdash (\lambda(x : A) \mapsto t) \Rightarrow (\lambda(x : A') \mapsto t') : (x : A) \to B \Rightarrow (x : A') \to B'}$$

Figure 1: Judgement rules for mappings